Coordinating Two AI Models Through GitHub

How the LB-AO musculoskeletal digital twin project moved its AI build-and-review workflow onto a shared repository, what happened when we tried it, and why we are keeping it

Project: Lower-Body Anatomical Ontology (LB-AO), groundwork for a musculoskeletal digital twin. Independent study, Misericordia University, Fall 2026.
Owner: Corbin Wright · **AI collaborators:** ChatGPT and Claude
Repository: github.com/Corb1nW/Independent-Study · **Records as of commit** 42ca56a (21 October 2026)
Prepared by Claude (Anthropic) at Corbin's request, from the repository's own records. Every claim below can be checked against the files and pull requests listed at the end. Claude is one of the two models described: this document reports the records rather than evaluating them independently.

In one paragraph

Two AI models take turns building and reviewing the project's code and data, and Corbin, the human owner, makes every decision. Until 20 October 2026 their work moved between them as zip files and chat messages. Decision **D-051** made a GitHub repository the meeting point instead. Every handoff is now a *commit* with a unique ID, every review names the exact commit it examined, and every approval is a recorded action by Corbin on a specific revision. A two-day pilot, eight pull requests, ran a complete cycle: build, review, fix, re-review, and an independent re-run of the tests by the second model. It met every pass criterion, and Corbin accepted D-051 on 21 October 2026. The next phase starts with the models' roles swapped.

1. The setup: two models, one owner

The project uses a deliberate division of labour (decision D-041). One model **builds**: it writes code, data and documentation. The other **reviews**: it checks the work, tries to break it, and reports findings. Corbin **decides**: model agreement is only a recommendation, and nothing becomes accepted without his approval or an explicit delegation. Two files hold the project's memory: `docs/DECISIONS.md`, which records every decision, who proposed it, how it is defended and who approved it, and `PROJECT_STATE.md`, which records what exists, what has been verified and what happens next.

For Phase A (fixing integrity defects in the ontology and its validator), Claude built and ChatGPT reviewed. For Phase B (a first hip-only simulation), the roles are swapped.

2. The problem with passing files around

Before D-051, a handoff meant downloading the repository as a zip, uploading it to the other model's chat, and often recreating files from chat output. On 20 October Corbin proposed giving both models direct access to the project files instead, to “save time and energy of downloading and sharing and then recreating files.” The time cost was the visible problem. Three less visible ones mattered as much:

- **No fixed snapshot.** A reviewer could not prove which version it had examined, and a file could change between a test run and the report about it.
- **Transcription risk.** A file regenerated from a chat message can differ silently from the one that was reviewed.
- **Scattered approvals.** Corbin's decisions lived in separate chat transcripts, which made them hard to cite.

3. Choosing the mechanism

Taken literally, Corbin's idea was for both models to work in one shared folder on his computer. Claude proposed a shared **git remote** instead. Git records every version permanently and gives each one a unique identifier, so that is what a review should point to. The options compared:

Option	Fixed snapshot a review can cite	Version history	Safe with two editors	Verdict
Shared local folder	No: files change underneath a review	No	No: silent overwrites	Allowed as a way of *working*, never as the handoff
Shared cloud folder	No	Limited	No	Rejected
Zip uploads (status quo)	Only if the zip includes git history	Only with git history	n/a	Kept as the fallback
Shared git remote (GitHub)	**Yes: the commit ID (SHA)**	**Yes, complete**	**Yes: separate branches**	**Chosen**

A **commit SHA** such as `4abb4f1995956d926959ac5ef0f0f68656619ee6` is a fingerprint of the entire project at one moment: every file, byte for byte. If two people name the same SHA, they are looking at exactly the same thing.

4. How the decision itself was made

D-051 went through the same build-and-review process it describes, and the review changed it substantially.

When (2026)	Step	Record
20 Oct, 13:11	Corbin proposes direct file access for both models	Session transcript, quoted in D-051
20 Oct	Claude drafts D-051 revision 1: a shared git remote, pull requests, owner-only merges	docs/reviews/…Claude_Proposal_D-051.md
20 Oct	ChatGPT reviews it: *conditional approval with amendments*	docs/reviews/…ChatGPT_Review_of_D-051.md

When (2026)	Step	Record
20 Oct	Claude's revision 2 adopts all amendments, narrowing one	docs/reviews/…Claude_Revised_Proposal_D-051.md
20 Oct, 14:40	Corbin approves running a **pilot** (Q9). The decision itself waits for the pilot result	DECISIONS.md §4
21 Oct, 09:10	Pilot passed. Corbin **accepts D-051** (Q15)	DECISIONS.md §4, Part V

ChatGPT's amendments are the reason the workflow is lean rather than bureaucratic:

- **Proportionate process.** Code, data and the substance of decisions get an independent review. Pure wording and formatting fixes take a lighter path, but still need the owner's merge.
- **One authoritative review record.** A review exists in one place, not copied into three.
- **Approval names a specific revision.** Approving one commit never silently covers commits added later.
- **Software verification is not scientific validation.** Passing tests show the code behaves as specified. They say nothing about whether the anatomy or biomechanics is right.
- **Test tool access; don't assume it.** Each model's actual GitHub abilities had to be demonstrated before the process relied on them. This mattered a great deal (Section 6).

5. The workflow

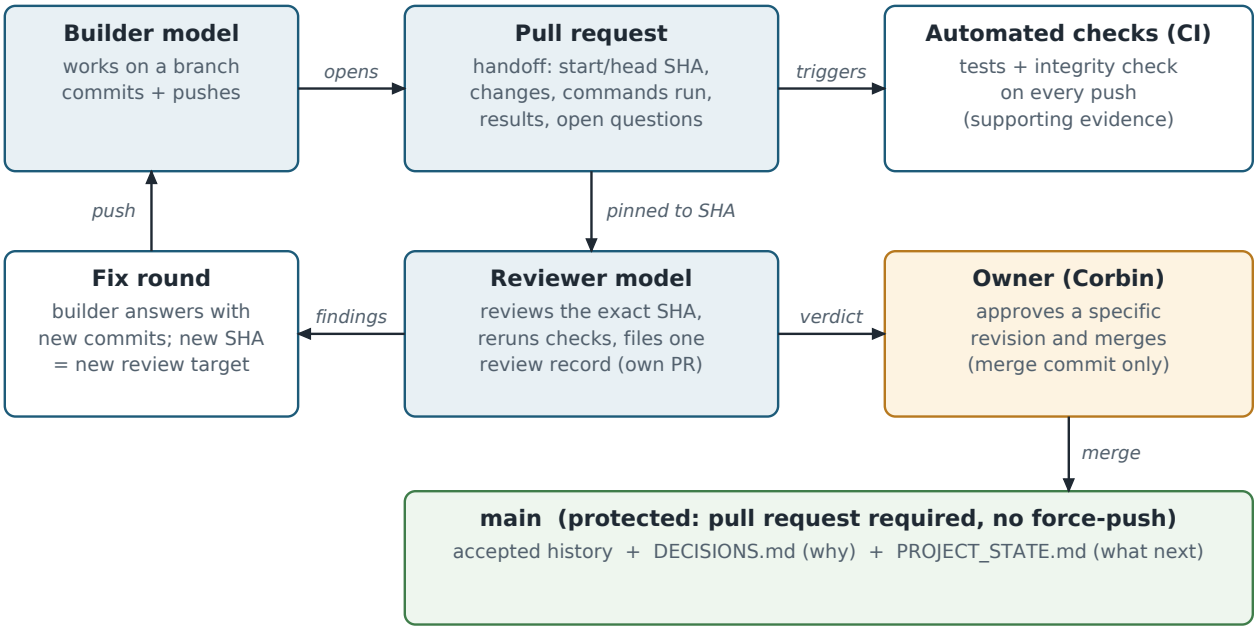


Figure 1. One build-and-review cycle under D-051. Either model can hold either role.

The essential rules, condensed from D-051's thirteen items:

- **One canonical repository.** Local folders are working copies.
- **Only Corbin integrates into `main`.** A GitHub ruleset blocks direct pushes, deletion and history rewriting on `main`, and allows only “merge commit” merges, so imported history is never flattened.

- **Work happens on branches** named after phase, model and topic (e.g. `phase-b/chatgpt-d037`). Each pull request description carries the handoff: start and head SHA, what changed, the commands run and their results, and open questions.
- **Reviews are pinned to a full SHA** and state plainly what the reviewer executed and what it only read. A model reviewing its own work does not count as independent verification.
- **Only pushed commits count as snapshots.** Uncommitted work is never "the reviewed state."
- **Fallback:** when a model can't reach GitHub, it gets a GitHub archive of a named commit. The handoff says which method was used.

6. The pilot: what actually happened

The pilot used real work: ChatGPT's review of Claude's Phase A implementation. Times are US Eastern.

PR	Author	What it did	Merged
#1	Claude	Imported the project's existing git history into Corbin's course repository with one merge commit, so no existing commit ID changed. Added a test configuration the combined layout needed.	20 Oct 15:08
—	Corbin	Created the ruleset protecting `main`.	21 Oct 08:24
#2	ChatGPT	Phase A review of commit 4abb4f1: **AMEND**, five findings (three medium, two low). Execution was not possible yet.	08:48
#3	Claude	Reproduced findings A1–A3 (each bad record was accepted), fixed A1–A4, added 15 tests and automated checks (CI).	08:49
#5	ChatGPT	Re-review at 29e256e: **AGREE** with the fixes; flagged stale records; test execution **blocked** (see below).	09:01
#4	Claude	Recorded Corbin's answers and the re-review results.	09:01
#7	ChatGPT	**Independent execution:** 80 tests passed, integrity check passed, 9 of 9 malformed-data probes rejected, all run by ChatGPT itself.	09:12
#6	Claude	Owner rulings on physical bounds; verification statuses upgraded; D-051 and D-052 accepted.	09:13
#8	Claude	Phase B roles recorded: ChatGPT builds, Claude reviews.	09:16

The execution problem and its fix. ChatGPT could read any commit through its GitHub connector, push branches and open pull requests. Its code sandbox, though, could reach neither GitHub nor the Python package index, and one required package (pint, for physical units) was missing. ChatGPT reported its first attempt honestly as *blocked*; it did not work around the gap. Claude built a bundle of the official package files, checked each file's SHA-256 fingerprint against the one PyPI publishes, and tested the bundle offline with ChatGPT's exact package versions. Corbin uploaded the bundle with a GitHub archive of the commit. ChatGPT then installed it, verified the fingerprints, matched three files against GitHub's records of that commit, and ran every check itself.

7. What the pilot taught us

- **Cross-model review finds real defects.** Three of ChatGPT's five findings were demonstrated to let bad records through. The 15 new tests were shown to catch them: run against the old code, the 11 tests aimed at the defects fail, and the 4 acceptance tests pass.
- **Status labels stay honest.** Results moved from “reported by author” to “independently checked (software)” only when the other model ran them. One check, which needs full git history, still carries the author-reported label, because nobody else has run it.
- **The records police themselves.** ChatGPT caught out-of-date state files. A duplicated review copy was removed so that one authoritative record remained. A formatting slip by Claude was caught and fixed in the open.
- **Process slips are visible and recoverable.** One pull request was merged before its re-review. The log recorded that, and the decision stayed “Proposed” until the review and the owner's acceptance came in.
- **Tooling surprises surface early.** Claude's session could not push git tags, so tags were dropped; commit IDs do the same job. A test tool that silently checked the wrong code after a refactor was found and changed to fail loudly.

8. The case for GitHub

Reproducibility. A commit ID pins code, data and documentation together. Anyone (a peer, an advisor, a third model) can retrieve that exact state and rerun the checks. This is the project's own rule applied to its process: the simulation must be able to say what number it used, and why.

Verification you can check. Independent review only means something if both parties examined the same thing. SHA-pinned reviews make that provable, and it is what let ChatGPT's independent run confirm Claude's results rather than merely echo them.

Human authority, enforced and traceable. AI models propose; the owner decides. Branch protection makes `main` change only through a reviewed pull request that Corbin merges, and each approval is tied to a specific revision and quoted word for word in the decision log.

Better reviews. A pull request shows exactly what changed, line by line. Reviews are durable files, not chat messages that scroll away.

Less overhead, fewer errors. No downloading, re-uploading or recreating files, so no transcription mistakes.

An audit trail for academic work. The history shows who proposed, built, reviewed and approved each change, and keeps human and model contributions distinguishable. That matters when AI-assisted work is assessed.

Roles are interchangeable. Phase B swaps builder and reviewer with no change to the process. Another model could join under the same rules.

Standard, free and transferable. Git, pull requests and continuous integration are everyday software-engineering practice, not a bespoke system.

9. Limitations and open risks

These are recorded in D-051 and in the project state file, not hidden:

- **Shared account.** Both models act through Corbin's GitHub account, so GitHub's author labels cannot tell them apart. Every review and handoff therefore names its author in the text. “Only Corbin merges” is enforced against direct pushes, but between Corbin and the models it remains a rule followed, not a technical barrier.
- **Public repository.** Corbin chose to keep the repository public, so anyone can read it. Its full history was scanned for credentials before the import, and none were found.
- **Uneven tool access.** ChatGPT's test runs depend on owner-supplied archives and offline packages, and tools that need git history cannot run in its sandbox.
- **CI is supporting evidence only.** Automated checks run in a third environment, but they do not replace a reviewer reproducing results.
- **Scope of verification.** Every check here concerns software behaviour. None of it establishes anatomical, biomechanical or clinical validity, which needs sources read and judged by people.
- **Process weight.** ChatGPT's guiding principle for the workflow applies: “It must remain simpler than the research process it supports.”

10. Doing this yourself

Step	What to do
1	Put the project in a GitHub repository and scan its full history for passwords or keys before sharing it.
2	Protect `main`: require pull requests, block force-pushes and deletion, and allow merge commits only.
3	Connect each AI model's GitHub integration, then *test* what each can actually do: read a commit, check one out, run the tests, push a branch, open a PR. Write the results down.
4	Agree on a handoff format: start and head SHA, changes, commands run with results, open questions.
5	Have reviews name the exact SHA and separate what was executed from what was only read.
6	Keep a decision log and a short project-state file in the repository, and update both in the same pull requests.
7	Run a small pilot on real work before relying on the process, and keep a fallback (commit archives).

Records referenced

All in github.com/Corb1nW/Independent-Study:

- docs/DECISIONS.md: D-041 (multi-model protocol), D-042 to D-048 (record-keeping rules), D-051 (this workflow, with Amendments 1–3), D-052 (Phase A review response); §4 approval record Q9–Q17 and owner actions A-1 to A-4
- PROJECT_STATE.md: capability table, pilot pass criteria, validation results
- docs/reviews/2026-10-20_Claude_Proposal_D-051.md, 2026-10-20_ChatGPT_Review_of_D-051.md, 2026-10-20_Claude_Revised_Proposal_D-051.md
- docs/reviews/2026-10-21_ChatGPT_Phase_A_Review_4abb4f1.md, 2026-10-21_Claude_Response_to_Phase_A_Review.md, 2026-10-21_ChatGPT_ReReview_D052_29e256e.md, 2026-10-21_ChatGPT_D052_Execution_Addendum_29e256e.md
- Pull requests #1 to #8; .github/workflows/tests.yml